

# Módulo 5: Superficie del Servidor

## T13: Headers HTTP y securityheaders.com

**Carlos Rodríguez Contreras** · GitHub: karrocon

 *CSP, HSTS y las 6 cabeceras que protegen a tus usuarios*

# Objetivos

---

- Entender qué protege cada cabecera de seguridad HTTP
- Identificar cabeceras faltantes en VulnApp con DevTools y securityheaders.com
- Configurar `helmet.js` para añadir todas las cabeceras de seguridad
- Conseguir una puntuación A en securityheaders.com

# Las 6 cabeceras de seguridad esenciales

| Header                    | Protege contra              | Valor recomendado                        |
|---------------------------|-----------------------------|------------------------------------------|
| Strict-Transport-Security | Downgrade a HTTP            | max-age=31536000; includeSubDomains      |
| Content-Security-Policy   | XSS, inyección de recursos  | default-src 'self'                       |
| X-Frame-Options           | Clickjacking                | DENY o SAMEORIGIN                        |
| X-Content-Type-Options    | MIME sniffing               | nosniff                                  |
| Referrer-Policy           | Filtración de URLs          | strict-origin-when-cross-origin          |
| Permissions-Policy        | Acceso a APIs del navegador | camera=(), microphone=(), geolocation=() |

# ¿Qué pasa sin cabeceras de seguridad?

VulnApp sin helmet:

```
HTTP/1.1 200 OK
Content-Type: application/json
X-Powered-By: Express      ← Revela tecnología
                          ← Sin CSP, HSTS, ni X-Frame-Options
```

💀 **Sin CSP:** un XSS puede cargar scripts de cualquier dominio externo.

**Sin X-Frame-Options:** la app puede ser embebida en un iframe (clickjacking).

**Sin HSTS:** la primera conexión puede ser HTTP (SSL stripping).

# Content-Security-Policy (CSP) – en detalle

Content-Security-Policy:

```
default-src 'self';  
script-src 'self' https://cdn.trusted.com;  
style-src 'self' 'unsafe-inline';  
img-src 'self' data: https;;  
connect-src 'self' https://api.example.com;  
frame-ancestors 'none';
```

| Directiva       | Controla                      |
|-----------------|-------------------------------|
| default-src     | Fallback para todo            |
| script-src      | De dónde se cargan scripts    |
| style-src       | De dónde se cargan estilos    |
| img-src         | Orígenes de imágenes          |
| frame-ancestors | Quién puede embeber la página |

# helmet.js — la solución rápida para Express

```
import helmet from 'helmet';

app.use(helmet());
// Añade automáticamente:
// - X-Content-Type-Options: nosniff
// - X-Frame-Options: SAMEORIGIN
// - Strict-Transport-Security
// - X-DNS-Prefetch-Control
// - Referrer-Policy
// Elimina: X-Powered-By

// CSP personalizado:
app.use(helmet.contentSecurityPolicy({
  directives: {
    defaultSrc: ["'self'"],
    scriptSrc: ["'self'"],
    styleSrc: ["'self'", "'unsafe-inline'"],
  }
}));
```

# securityheaders.com – verificación rápida

| Nota | Significado                                          |
|------|------------------------------------------------------|
| A+   | Todas las cabeceras presentes y bien configuradas    |
| A    | Muy buena configuración                              |
| B-D  | Faltan cabeceras importantes                         |
| F    | Sin cabeceras de seguridad (como VulnApp sin helmet) |

✓ **Objetivo del lab:** pasar VulnApp de F a A con `helmet` + CSP custom.

# CSP Report-Only — despliegue gradual

```
# Fase 1: Solo reportar (no bloquea nada)
Content-Security-Policy-Report-Only:
  default-src 'self';
  report-uri /csp-violations;

# Fase 2: Una vez validado que no rompe nada → activar
Content-Security-Policy:
  default-src 'self';
  report-uri /csp-violations;
```

```
// Endpoint para recibir reports de violaciones CSP
app.post('/csp-violations', express.json({ type: 'application/csp-report' })), (req, res) => {
  console.warn('CSP Violation:', req.body['csp-report']);
  res.status(204).end();
});
```

💡 Usa **Report-Only** durante 1-2 semanas antes de activar CSP en producción. Te avisa de qué se rompería.



# Clickjacking — el ataque que X-Frame-Options previene

```
<!-- Página del atacante: evil.com -->
<style>
  iframe { opacity: 0; position: absolute; top: 0; left: 0; width: 100%; height: 100%; }
  .decoy { position: relative; z-index: -1; }
</style>
<div class="decoy"><h1>¡Haz click para ganar un iPhone!</h1></div>
<iframe src="https://bank.com/transfer?to=attacker&amount=1000"></iframe>
```

El usuario cree que clicka en "ganar iPhone" pero realmente clicka en el botón de transferencia del banco (invisible).

## Defensa:

```
X-Frame-Options: DENY
# 0 con CSP moderno:
Content-Security-Policy: frame-ancestors 'none';
```

## Lab T13: De F a A en securityheaders.com

**Objetivo:** configurar todas las cabeceras de seguridad en VulnApp

**Setup:** `cd VulnApp && npm start`

1. Visita `http://localhost:3000` con DevTools → Network → mira las cabeceras de respuesta
2. Anota qué cabeceras de seguridad faltan
3. Instala `helmet` y añádelo como middleware en `src/app.ts`
4. Reinicia y verifica las cabeceras de nuevo — ¿qué ha cambiado?

 **Herramienta:** [securityheaders.com](https://securityheaders.com) — pega tu URL y obtén un análisis gratuito